

รายงานแลป OLTP OLAP และ Pivot

ชื่อและรหัสนิสิต นายธนากร ดวงจันทร์ 67160341 กลุมเรียน 2

1 OLTP

โค้ด / ผลรันสองรอบ/ คำอธิบาย สถานะ และ ETL

```
(.venTESTX!) PS C:\lab\labs> python lab.py
Prepared data already exists. Use --reset only to discard lab data changes.
(.venTESTX!) PS C:\lab\labs> python oltp_demo.py
Before: [('01004', 'PENDING')]
Updated rows: 1
After: [('01004', 'PAID')]
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python oltp_demo.py
Before: [('01004', 'PENDING')]
Updated rows: 1
After: [('01004', 'PAID')]
(.venTESTX!) PS C:\lab\labs> python oltp_demo.py
Before: [('01004', 'PAID')]
Updated rows: 0
After: [('01004', 'PAID')]
(.venTESTX!) PS C:\lab\labs>
```

คำตอบ

จำนวนแถวที่แก้ไข: รอบแรกแก้ไขได้ 1 แถว ส่วนรอบสองแก้ไขได้ 0 แถว

เหตุผลที่ต้องมีเงื่อนไข status: เพื่อป้องกันการอัปเดตข้อมูลซ้ำซ้อน และป้องกันสถานะถูกเปลี่ยนแปลง

ผิดพลาด หากออเดอร์นั้นไม่ได้อยู่ในสถานะ PENDING อยู่ก่อนแล้ว

เหตุผลที่เป็น OLTP: เพราะเป็นการทำธุรกรรม (Transaction) ประจำวันที่เน้นการเปลี่ยนแปลง/อัปเดตสถานะข้อมูล ณ เวลาปัจจุบันแบบ Real-time แม้จะมี SELECT อ่านสถานะ แต่ก็เป็น การอ่าน

เพื่อตรวจสอบข้อมูลก่อนดำเนินการทำธุรกรรม ไม่ใช่การดึงไปวิเคราะห์ภาพรวมย้อนหลัง

การเปลี่ยน oltp.db กับ warehouse.db: ไม่ทำให้อยอดใน warehouse.db เปลี่ยนทันที เนื่องจากฐาน

ข้อมูล OLTP (ระบบหนาราน) และ Data Warehouse แยกออกจากกันเพื่อไม่ให้กระทบประสิทธิภาพ

การทำงานของกันและกัน

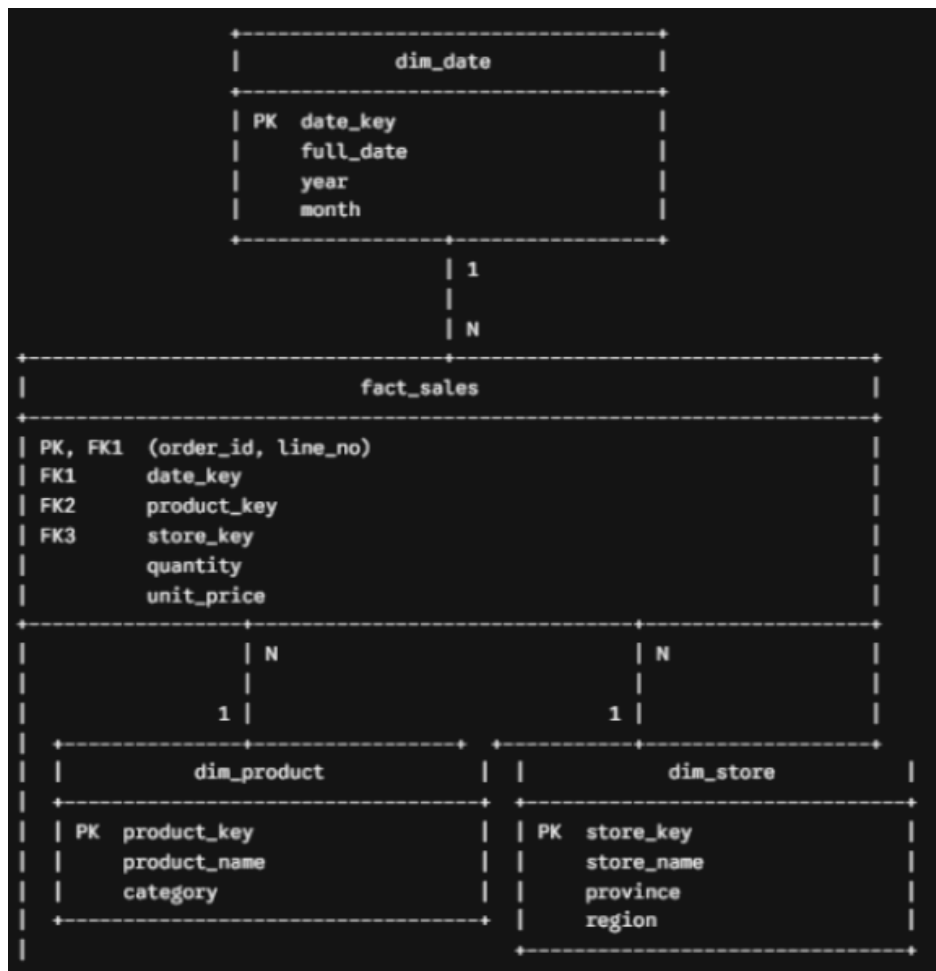
กระบวนการที่ใช้เชื่อมข้อมูล: ในระบบจริงควรใช้กระบวนการ ETL (Extract, Transform, Load) หรือ

CDC (Change Data Capture) เพื่อดึงและแปลงข้อมูลจากระบบ OLTP เข้าสู Data Warehouse

ตามรอบเวลาที่กำหนด (เช่น รายวัน/รายชั่วโมง)

2 Grain และ Star Schema

แผนภาพ/PK/FK/Dimensions/Measures/Hierarchy/ผล q01



ผลตรวจที่ได้

จำนวนรายการ 8 จำนวนออเดอร์ 6 จ

จำนวนชิ้น: 23

ยอดขายรวม: 1390 บาท

```
FileNotFoundError: [Errno 2] No such file or directory: 'q01.sql'
(.venvTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q01.sql
line_count      order_count      units      revenue
8              6              23          1390
(.venvTESTX!) PS C:\lab\labs>
```

คำอธิบาย Grain และมิติ

Grain (1 แถวใน fact_sales หมายถึง): รายการสินค้าแต่ละลำดับ ที่ถูกซื้อภายใน 1 ออเดอร์

ตัวอย่าง O1001: ออเดอร์ O1001 มี 2 แถวรายการ ได้แก่ line_no 1 (ชื่อ Tea 2 ชิ้น) และ line_no 2 (ชื่อ Cookie 1 ชิ้น) ซึ่งรวมกันเป็น 1 ออเดอร์ แต่คิดเป็น 2 รายการแยกใน fact_sales

3 Dimensions:

dim_date (มิติเวลา)

dim_product (มิติสินค้า)

dim_store (มิติสถานที่/สาขา)

2 Measures:

quantity (จำนวนชิ้นที่ขาย)

amount (ยอดเงินขาย = quantity * unit_price)

Hierarchy ของเวลา: Year -> Month -> Full Date (ปี -> เดือน -> วัน)

Hierarchy ของสถานที่: Region -> Province -> Store Name (ภาค -> จังหวัด -> ชื่อสาขา)

เหตุผลที่ unit_price ไม่ควรนำมา SUM: เพราะ unit_price เป็นราคาต่อหน่วย ณ การขาย ไม่ใช่ยอด

สะสม หากนำมาบวกกันโดยตรงจะไม่สะท้อนยอดขายจริงเนื่องจากไม่ได้คูณกับจำนวนชิ้น (quantity)

ยอดขายรวมที่ถูกต้องจึงต้องใช้ SUM(amount) หรือ SUM(quantity * unit_price) เท่านั้น

3 OLAP

ผล q02-q07 และ q12 พร้อ มคำอธิบาย Operation และ WHERE/HAVING

ก. ความแตกต่างระหว่าง q03 กับ q04:

q03 เป็นการ Add Dimension โดยนำมิติสถานที่ (province) เขามารวมจำแนกกับ มิติเวลา ทำให้เห็น

ยอดขายแยกตามจังหวัดในแต่ละเดือน

q04 เป็นการ Drill-down มิติเวลา จากระดับ "เดือน" ลงไปสู่ระดับ "วัน" (full_date) รวมกับทำ Slice

กรองเอาเฉพาะเดือนกันยายน

ข. ตำแหน่งการกรองใน q07 (WHERE vs HAVING):

WHERE month = '2026-09': กรองระดับแถวข้อมูล (Row-level) ก่อน นำไป จัดกลุ่ม

HAVING SUM(amount) > 400: กรองระดับกลุ่ม (Group-level) หลังจากที่มีการ จัดกลุ่มและคำนวณ

ผลรวม SUM(amount) เรียบร้อยแล้ว

ค. การตรวจสอบความถูกต้อง (q12 เทียบกับ q03):

ใน q03 ยอดขายรวมของ Bangkok เดือน 2026-09 ได้ 540 บาท

ใน q12 รายการขายละเอียดของ Bangkok เดือน 2026-09 มี 2 รายการ คือ 300 + 240 = 540 บาท

ซึ่งตรงกันพอดี ถือว่าข้อมูลถูกต้อง

```
FileNotFoundError: [Errno 2] No such file or directory: 'q01.sql'
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q01.sql
line_count    order_count    units    revenue
8            6            23        1390
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q02.sql
month    revenue
2026-08  490
2026-09  900
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q03.sql
month    province    revenue
2026-08  Bangkok    310
2026-08  Chonburi    180
2026-09  Bangkok    540
2026-09  Chonburi    360
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q04.sql
full_date    revenue
2026-09-09    360
2026-09-10    300
2026-09-11    240
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q05.sql
province    revenue
Bangkok    540
Chonburi    360
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q06.sql
month    category    province    revenue
2026-09  Drink    Bangkok    300
2026-09  Drink    Chonburi    200
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q07.sql
province    revenue
Bangkok    540
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q12.sql
order_id    line_no    product_name    quantity    amount
01005    1    Tea    6    300
01006    1    Cookie    3    240
```

ข้อ Operation ผลลัพธ์

๑

q0 Roll-up

2

ส.ค. 2026 = 490ก.ย.
2026 = 900

คำอธิบาย

สรุปยอดขายจากระดับรายวันให้เป็น ระดับเดือน ทำให้เห็นยอดรวมของแต่ละเดือน

q0 Drill-down /	ส.ค.: Bangkok 310 ,	เจาะลึกข้อมูลจากระดับเดือน โดย
3 Add	Chonburi 180 ก.ย.:	เพิ่มมิติ จังหวัด เพื่อดูยอดขาย
Dimension	Bangkok 540 ,	แยกตามพื้นที่
	Chonburi 360	
q0 Drill-down +	9 ก.ย. = 360 10 ก.ย. =	เจาะลึกจากระดับเดือนลงมาเป็น
4 Slice	300 11 ก.ย. = 240	รายวัน และเลือกดูเฉพาะเดือน
		กันยายน 2026
q0 Slice	Bangkok 540 Chonburi	เลือกดูเฉพาะข้อมูลของ เดือน
5	360	กันยายน 2026 โดยตัดข้อมูล
		เดือนอื่นออก
q0 Dice	ก.ย. Drink: Bangkok	กรองข้อมูลหลายมิติพร้อมกัน
6	300 ก.ย. Drink:	ได้แก่ เดือน + หมวดสินค้า +
	Chonburi 200	จังหวัด
q0 Slice +	Bangkok 540	เลือกเฉพาะเดือน กันยายน แล้ว
7 Aggregation		รวมยอดขายตามจังหวัด จากนั้นใช้
n + Filter		HAVING เลือกจังหวัดที่ยอดขาย
		รวมมากกว่า 400
q1 Drill-through	O1005: Tea, Qty 6 ,	เจาะจากข้อมูลสรุปลงไปดู ราย
2 h	Amount 300 O1006:	ละเอียดรายการขายจริง
	Cookie, Qty 3 , Amount	(Transaction Level) ของ
	240	Bangkok ในเดือนกันยายน

4 Pivot

ผล q08/P1/P2/Drink และก่อกำหนดให้ mean พร้อมผล assert

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q08.sql
month   Bangkok Chonburi
2026-08 310     180
2026-09 540     360
(.venTESTX!) PS C:\lab\labs>
```

```

--- P1: Pivot Province x Month ---
month      2026-08  2026-09  Total
province
Bangkok      310      540      850
Chonburi     180      360      540
Total        490      900     1390

--- P2: Pivot September (Category x Province) ---
province Bangkok  Chonburi
category
Drink           300    200
Snack           240    160

```

ก่อน

```

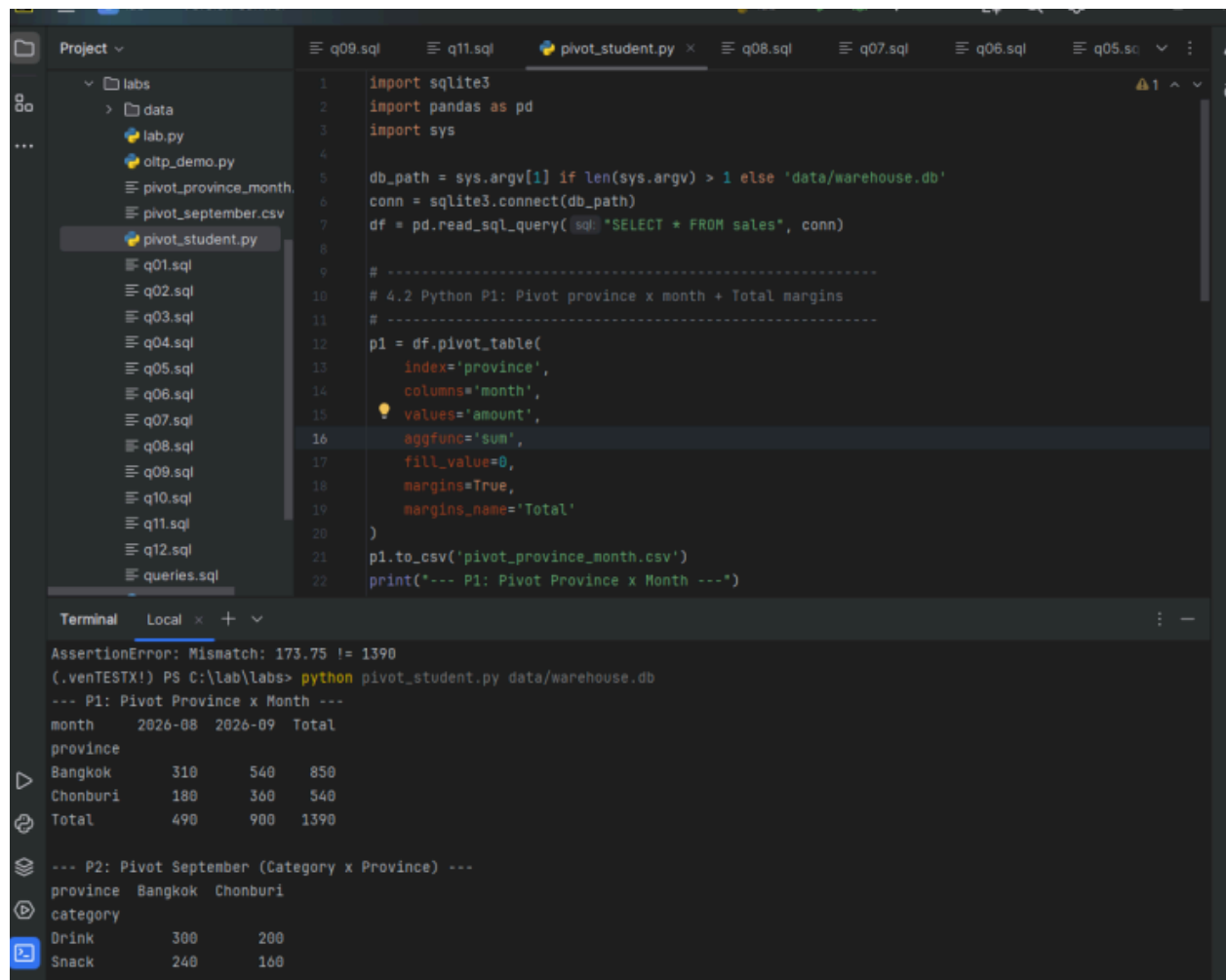
Project: lab
File Explorer:
- labs
  - data
    - lab.py
    - oitp_demo.py
    - pivot_province_month.csv
    - pivot_september.csv
    - pivot_student.py
    - q01.sql
    - q02.sql
    - q03.sql
    - q04.sql
    - q05.sql
    - q06.sql
    - q07.sql
    - q08.sql
    - q09.sql
    - q10.sql
    - q11.sql
    - q12.sql
    - queries.sql
  - pivot_student.py
Code Editor:
1 import sqlite3
2 import pandas as pd
3 import sys
4
5 db_path = sys.argv[1] if len(sys.argv) > 1 else 'data/warehouse.db'
6 conn = sqlite3.connect(db_path)
7 df = pd.read_sql_query("SELECT * FROM sales", conn)
8
9 # -----
10 # 4.2 Python P1: Pivot province x month + Total margins
11 # -----
12 p1 = df.pivot_table(
13     index='province',
14     columns='month',
15     values='amount',
16     aggfunc='sum',
17     fill_value=0,
18     margins=True,
19     margins_name='Total'
20 )
21 p1.to_csv('pivot_province_month.csv')
22 print("--- P1: Pivot Province x Month ---")
Terminal:
*****
AssertionError: Mismatch: 173.75 != 1390
(.venv\TESTX\PS C:\lab\labs> python pivot_student.py data/warehouse.db
--- P1: Pivot Province x Month ---
month      2026-08  2026-09  Total
province
Bangkok      155.0    270.0   212.50
Chonburi      90.0    180.0   135.00
Total       122.5    225.0   173.75

--- P2: Pivot September (Category x Province) ---
province Bangkok  Chonburi
category
Drink           300    200
Snack           240    160
Traceback (most recent call last):
  File "C:\lab\labs\pivot_student.py", line 46, in <module>

```

เมื่อไม่มีการกำหนด `aggfunc` ระบบจะใช้ค่าเฉลี่ย (`mean`) เป็นค่าเริ่มต้น ทำให้ผลลัพธ์เป็น **270.0** และเกิด `AssertionError`

หลัง



```
1 import sqlite3
2 import pandas as pd
3 import sys
4
5 db_path = sys.argv[1] if len(sys.argv) > 1 else 'data/warehouse.db'
6 conn = sqlite3.connect(db_path)
7 df = pd.read_sql_query('SELECT * FROM sales', conn)
8
9 # -----
10 # 4.2 Python P1: Pivot province x month + Total margins
11 # -----
12 p1 = df.pivot_table(
13     index='province',
14     columns='month',
15     values='amount',
16     aggfunc='sum',
17     fill_value=0,
18     margins=True,
19     margins_name='Total'
20 )
21 p1.to_csv('pivot_province_month.csv')
22 print('--- P1: Pivot Province x Month ---')
```

Terminal

```
AssertionError: Mismatch: 173.75 != 1390
(.venv\TESTX\PS C:\lab\labs> python pivot_student.py data/warehouse.db
--- P1: Pivot Province x Month ---
month      2026-08  2026-09  Total
province
Bangkok      310      540      850
Chonburi     180      360      540
Total        490      900     1390

--- P2: Pivot September (Category x Province) ---
province Bangkok Chonburi
category
Drink         300      200
Snack         240      160
```

กำหนด `aggfunc='sum'` เพื่อรวมยอดขายจริง ทำให้ได้ผลลัพธ์ **540** และผ่าน `Assert` ได้สำเร็จ

ทำไม Bangkok เดือนกันยายนจึงได้ 270?

เมื่อเราลบ `aggfunc='sum'` ออกจากฟังก์ชัน `pivot_table()` Pandas จะใช้ค่าเริ่มต้น (Default) เป็น `aggfunc='mean'` ซึ่งหมายถึงการหาค่าเฉลี่ย

ข้อมูลเดือนกันยายน (2026-09) ของสาขา Bangkok มีรายการขาย 2 รายการ คือ **300 บาท และ 240 บาท**

ดังนั้น Pandas จึงคำนวณค่าเฉลี่ยเป็น

$$(300 + 240) \div 2 = 270 \text{ บาท}$$

ค่า 270 แสดงอะไร?

ค่า **270 บาท** คือ ยอดขายเฉลี่ยต่อรายการ ของสาขา Bangkok ในเดือนกันยายน ไม่ใช่ยอดขายรวม

หากต้องการให้ผลลัพธ์เป็นยอดขายรวม ต้องกำหนด `aggfunc='sum'` ใน `pivot_table()` เพื่อให้ Pandas นำยอดขายทุกรายการมาบวกกัน

$$300 + 240 = 540 \text{ บาท}$$

ดังนั้นยอดขายรวมของ Bangkok ในเดือนกันยายนคือ **540 บาท** และ Grand Total รวมเป็น **1,390 บาท** ทำให้ผ่านการตรวจสอบ (`Assert`)

ผลรวมหลังกรอง Drink

ยอดรวมหลังกรองสินค้า **Drink** เท่ากับ **500 บาท**

มาจาก:

- Bangkok = 300 บาท
- Chonburi = 200 บาท

รวม = 500 บาท

โครงสร้าง Pivot Table

- Rows: `province`

- **Columns:** month
- **Values:** SUM(amount)

ความแตกต่างระหว่าง **sum** และ **mean**

- **sum** → นำยอดขายทุกรายการมาบวกกัน เพื่อหายอดขายรวมทั้งหมด
- **mean** → หาค่าเฉลี่ยของยอดขายแต่ละรายการ เหมาะสำหรับดูขนาดยอดขายเฉลี่ยต่อรายการ

คำถามชวนคิด

หากเพิ่มเดือนตุลาคม SQL ที่เขียน CASE เฉพาะเดือนสิงหาคมและกันยายน จะแสดงคอลัมน์ใหม่เองหรือไม่? แล้ว Pandas ที่ใช้ **columns="month"** ทำอย่างไร?

SQL (ข้อ 4.1):

จะ **ไม่สร้างคอลัมน์ใหม่ให้อัตโนมัติ** หากเขียน **CASE** ระบุเฉพาะเดือนสิงหาคมและกันยายน ดังนั้นต้องแก้ไข SQL โดยเพิ่มเงื่อนไขของเดือนตุลาคม เช่น

```
SUM(CASE WHEN month = '2026-10' THEN amount ELSE 0 END)  
AS "2026-10"
```

Pandas (**columns='month'**):

จะสร้างคอลัมน์ใหม่ให้อัตโนมัติ ตามค่าเดือนที่มีอยู่ใน DataFrame โดยไม่จำเป็นต้องแก้ไขโค้ด

5. ตรวจสอบความถูกต้อง

ตรวจสอบผลลัพธ์ของ q09-q11 รวมถึงชนิดของ **Measure** และคำอธิบายเกี่ยวกับ **AOV** และ **JOIN** เพื่อยืนยันว่าผลลัพธ์ที่ได้ถูกต้องตามโจทย์

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q09.sql
month    revenue
2026-08  490
2026-09  900
ALL      1390
(.venTESTX!) PS C:\lab\labs>
```

หากนำผลลัพธ์จาก q09 ทุกแถว ได้แก่ **2026-08**, **2026-09** และ **ALL** มาทำ **SUM()** รวมกันอีกครั้ง จะทำให้เกิด **การนับยอดขายซ้ำซ้อน** เพราะแถว **ALL** เป็น ยอดรวมของข้อมูลรายเดือนอยู่แล้ว

ดังนั้นควรรวมเฉพาะแถว **2026-08** และ **2026-09** หรือใช้ค่าในแถว **ALL** โดยตรง เพื่อหลีกเลี่ยงการนับยอดขายซ้ำ

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q10.sql
revenue orders  aov    avg_line
1390      6         231.67 173.75
(.venTESTX!) PS C:\lab\labs>
```

AOV (Average Order Value): มีหน่วยเป็น บาทต่อออเดอร์ โดยคำนวณจาก ยอดขายรวม ÷ จำนวนออเดอร์ที่ไม่ซ้ำกัน (**COUNT(DISTINCT order_id)**)

AVG(amount): มีหน่วยเป็น บาทต่อรายการสินค้า (Line Item) โดยคำนวณจาก ยอดขายรวม ÷ จำนวนแถวทั้งหมด (**COUNT(*)**)

สรุป: AOV ใช้มูลค่าเฉลี่ยต่อ **ออเดอร์** ส่วน **AVG(amount)** ใช้มูลค่าเฉลี่ยต่อ

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q11.sql
source    total_rows  total_revenue
fact_sales (Before JOIN)      8      1390
sales view (After JOIN) 8      1390
```

ถ้าคีย์ซ้ำใน Dimension: เมื่อนำไป JOIN แบบ INNER JOIN จะทำให้แถวข้อมูลใน Fact ทวีคูณ ส่งผลให้ยอดขายรวม (SUM(amount)) สูงกว่าความเป็นจริง

ถ้า FK หายไป: แถวใน Fact ที่ไม่มีคีย์ตรงกับ Dimension จะถูก กรองทิ้ง เมื่อใช้ INNER JOIN ส่งผลให้จำนวนแถวและยอดขายรวม ลดลงจากความเป็นจริง

ภาพหน้าจอ Terminal: แสดงผลการรันคำสั่ง q01–q12 เพื่อใช้ตรวจสอบความถูกต้องของผลลัพธ์

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q01.sql
line_count    order_count    units    revenue
8             6              23       1390
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q02.sql
month    revenue
2026-08  490
2026-09  900
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q03.sql
month    province    revenue
2026-08  Bangkok     310
2026-08  Chonburi    180
2026-09  Bangkok     540
2026-09  Chonburi    360
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q04.sql
full_date    revenue
2026-09-09    360
2026-09-10    300
2026-09-11    240
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q05.sql
province      revenue
Bangkok 540
Chonburi      360
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q06.sql
month  category      province      revenue
2026-09 Drink    Bangkok 300
2026-09 Drink    Chonburi      200
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q07.sql
province      revenue
Bangkok 540
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q08.sql
month  Bangkok Chonburi
2026-08 310      180
2026-09 540      360
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q09.sql
month  revenue
2026-08 490
2026-09 900
ALL      1390
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q10.sql
revenue orders  aov    avg_line
1390      6      231.67 173.75
(.venTESTX!) PS C:\lab\labs>
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q11.sql
source  total_rows  total_revenue
fact_sales (Before JOIN)      8      1390
sales view (After JOIN) 8      1390
```

```
(.venTESTX!) PS C:\lab\labs> python query.py data/warehouse.db q12.sql
order_id      line_no product_name  quantity  amount
01005    1      Tea        6         300
01006    1      Cookie     3         240
(.venTESTX!) PS C:\lab\labs>
```

6. สรุปผล

6.1 ข้อค้นพบจากการวิเคราะห์ข้อมูล

1. ยอดขายในเดือนกันยายนเพิ่มขึ้นจากเดือนสิงหาคมอย่างชัดเจน

จากการวิเคราะห์พบว่าเดือนกันยายน 2026 มียอดขายรวม **900 บาท** ขณะที่เดือนสิงหาคมมียอดขาย **490 บาท** เพิ่มขึ้น **410 บาท** หรือประมาณ **83.67%** แสดงให้เห็นว่ายอดขายในเดือนกันยายนสูงกว่าเดือนก่อนอย่างชัดเจน

2. Bangkok มียอดขายรวมสูงกว่า Chonburi

เมื่อแยกข้อมูลตามจังหวัด พบว่า Bangkok มียอดขายรวม **850 บาท** แบ่งเป็นเดือนสิงหาคม **310 บาท** และเดือนกันยายน **540 บาท** ส่วน Chonburi มียอดขายรวม **540 บาท** ดังนั้น Bangkok จึงมียอดขายรวมมากกว่า Chonburi

6.2 ข้อจำกัดของข้อมูล

ชุดข้อมูลที่ใช้ในการวิเคราะห์มีขนาดค่อนข้างเล็ก โดยประกอบด้วยข้อมูลเพียง **8 รายการ** จาก **6 ออเดอร์** และครอบคลุมช่วงเวลาเพียง **2 เดือน คือ สิงหาคม-กันยายน 2026** จึงยังไม่เพียงพอสำหรับการวิเคราะห์แนวโน้มระยะยาว เช่น ฤดูกาลของยอดขาย หรือรูปแบบพฤติกรรมของลูกค้าในช่วงเวลาที่แตกต่างกัน

6.3 การใช้ AI ในการทำแลป

1. ตัวอย่าง Prompt ที่ใช้

Prompt 1 – การเขียน OLAP Query และ Pivot Table

"ช่วยอธิบายวิธีเขียน SQL ตามหลัก OLAP Operation และช่วยสร้างโค้ด `pivot_student.py` สำหรับทำ Pivot Table ด้วย Pandas"

Prompt 2 – การตรวจสอบแนวคิดและความถูกต้อง

"ช่วยอธิบายความแตกต่างระหว่าง WHERE กับ HAVING ใน q07 และ อธิบายความแตกต่างระหว่าง AOV กับ AVG(amount) ใน q10"

2. สิ่งที่คุณเรียนตรวจสอบและแก้ไขด้วยตนเอง

การแก้ไขตำแหน่งไฟล์และ Path ใน Terminal

ในระหว่างการทำงานพบว่าเมื่อใช้คำสั่ง `python query.py data/warehouse.db q01.sql` ระบบแจ้ง `FileNotFoundError` ผู้เรียนจึงตรวจสอบโครงสร้างไฟล์ผ่าน Project Tree ใน VS Code และพบว่าไฟล์ `q01.sql` อยู่ภายในโฟลเดอร์ `labs/data/` จึงย้ายไฟล์มาไว้ที่โฟลเดอร์หลัก `labs/` ให้อยู่ระดับเดียวกับ `query.py` หลังจากแก้ไขตำแหน่งไฟล์แล้วสามารถรันคำสั่งได้ตามปกติ

การปรับ Rows และ Columns ของ Pivot Table

ในขั้นแรก AI สร้าง Pivot Table โดยกำหนด `month` เป็นแถวและ `province` เป็นคอลัมน์ ซึ่งไม่ตรงกับข้อกำหนดของโจทย์ ผู้เรียนจึงตรวจสอบโจทย์อีกครั้งและปรับโค้ดใน `pivot_student.py` ให้ใช้ `province` เป็น Rows และ `month` เป็น Columns ด้วยคำสั่ง `index='province', columns='month'` รวมถึงปรับ `CASE WHEN` ใน `q08.sql` เพื่อให้ค่ารวม `total` แสดงผลถูกต้อง

การตรวจสอบ Foreign Key และ Data Integrity

ผู้เรียนตรวจสอบความถูกต้องของความสัมพันธ์ระหว่าง Fact และ Dimension ด้วยคำสั่ง `PRAGMA foreign_key_check;` ผ่าน SQLite Shell ผลการตรวจสอบไม่พบข้อผิดพลาดหรือ Orphan Key จึงยืนยันได้ว่าข้อมูลที่เชื่อมโยงระหว่างตารางสามารถ JOIN กันได้ครบถ้วน และไม่มีข้อมูลยอดขายสูญหายจากปัญหา Foreign Key